C++

Juha Yli-Hemminki

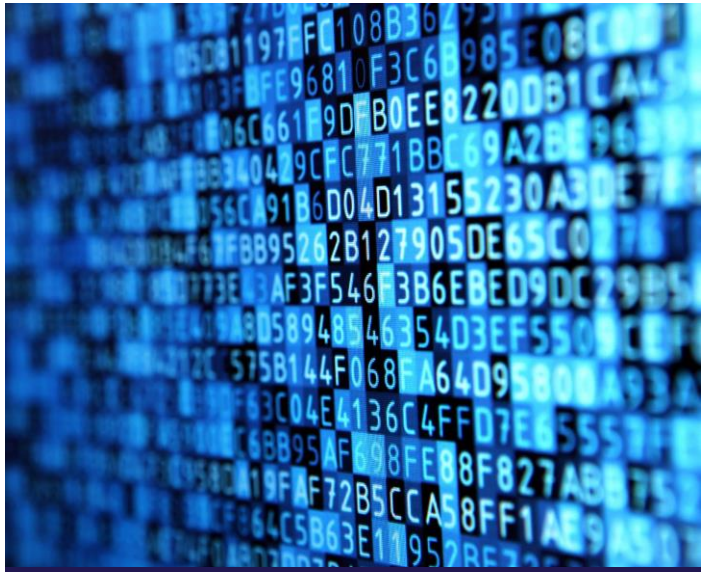
23.10.2025

SeAMK

SeAMK

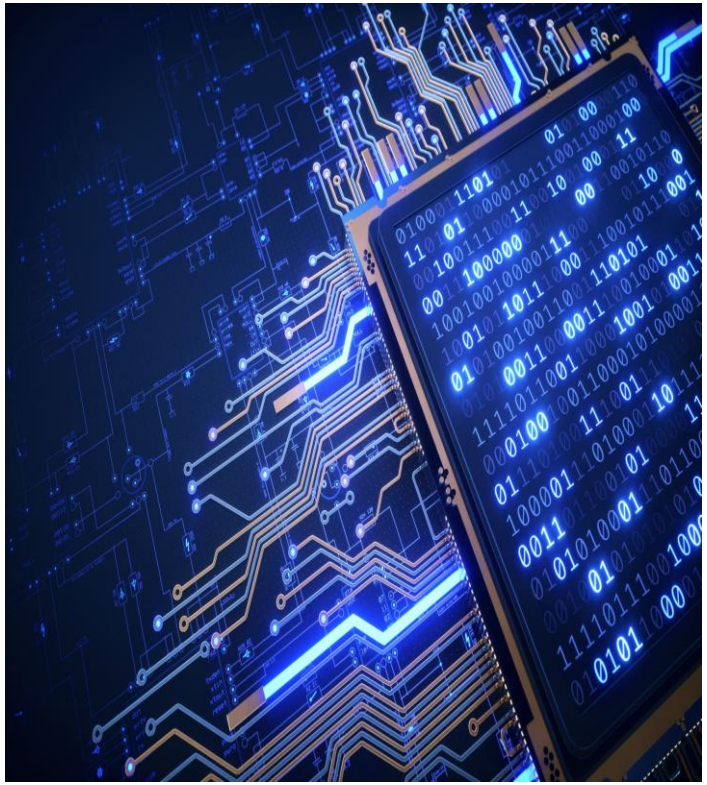
Course contents

- Compiler
- C++ type system
- Stream operators
- Header files
- Namespaces
- Pointers and references
- Classes and objects
- C++ container classes: Vector, List, Map
- Memory management and smart pointers
- Binary operators
- Generic programming and templates



SeAMK

Programming languages



- The purpose of programming languages is to act as an interface between humans and computers
- The goal of programming is to give the machine unambiguous operating instructions
- Without software, computers and smart devices do nothing
- Programming languages: C, C++, Java, C#, Python

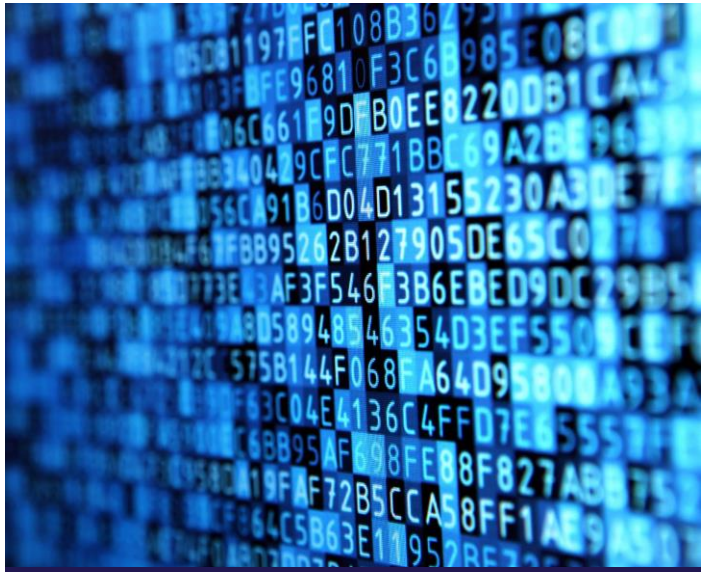
Programming languages

Pic: CC BY-SA 3.0/Bill Bertram/Wikipedia

- Based on the processor instruction set -> machine language
- In the beginning, programs were written in binary format -> difficult and error prone
- Simple commands such as calculation and transfer of the result to a memory location
- Symbolic machine code:
  - ADD A,B,@C vs.10110100 10110111

History of C++

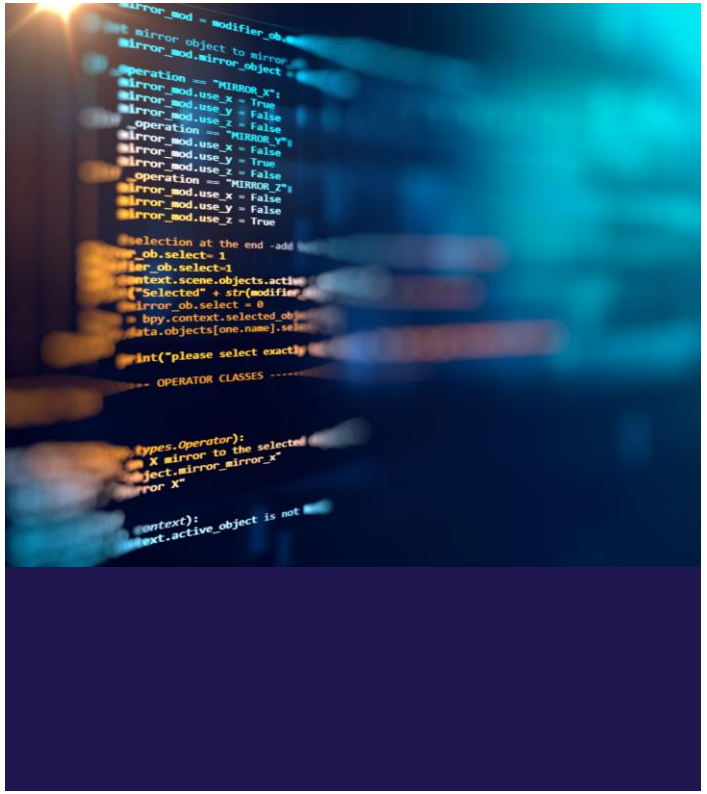
- In beginning of 1970 new efficient programming language was created: C
  - Dennis Ritchie created to develop Unix OS
  - Made programs portable to different processors as a source code. (Assembly code was processor dependent)
  - Simple, no support for object-oriented programming
  - Unix, Linux and old Windows versions are written in C



SeAMK

History of C++

- Development of C++ started in 1979
  - Bjarne Stroustrup
  - First named ”C with Classes”
  - Object oriented extensions to C
  - Name C++ has been used since year 1983
  - ISO standardized 1998, 2011 and every three years after that



Compiled language

- C++ source code is compiled to executable binary
  - Executable file is on target machines native language
  - Efficient and fast programs
  - Not portable as binary to different processor architecture
- Compare to interpreted languages such as Python that requires interpreter or bytecode languages as C# or Java that requires a runtime platform

SeAMK

Compiled language

source code

hello.py → **Interpreter** → 10011001001001 → CPU

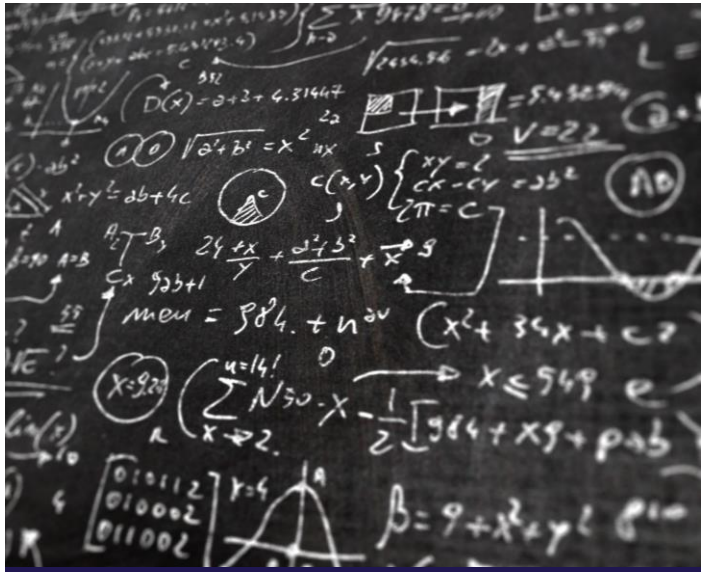
hello.cs → **Compiler** → bytecode → **Runtime** → 10011001001001 → CPU

hello.cpp → **Compiler** → hello.exe → 10011001001001 → CPU

SeAMK

Complex language

- No. It is high level language and meant to be human readable :)
- Focuses on concepts that are easy to understand such as objects and functions
- Made to handle complex structures and large systems
- Lots of control and responsibility for a programmer
- Lots of libraries available (Boost and some 3000+ other)



C language reserved keywords (32)

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
continue	for	signed	void
do	if	static	while
default	goto	sizeof	volatile
const	float	short	unsigned

SeAMK

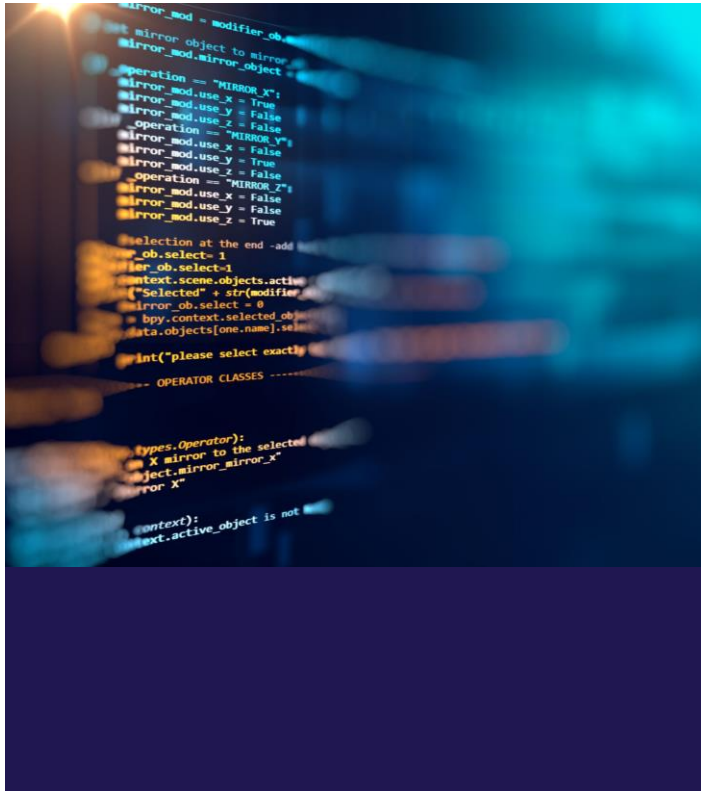
C++ language reserved keywords (95)

alignas (since C++11)
And
Asm
atomic_commit (TM TS)
auto(1)
Bitor
Break
Catch
char16_t (since C++11)
class(1)
concept (since C++20)
constexpr (since C++11)
Continue
co_return (coroutines TS)
decltype (since C++11)
delete(1)
double
else
explicit
extern(1)
float
friend
if
inline(1)
long
mutable(1)
new
not
nullptr (since C++11)
or
private
public
reinterpret_cast
return
signed
static
static_cast
switch
template
thread_local (since C++11)
true
typedef
typename
unsigned
virtual
volatile
while
xor_eq
alignof (since C++11)
and_eq
atomic_cancel (TM TS)
atomic_noexcept (TM TS)
bitand
bool
case
Char
char32_t (since C++11)
compl
const
const_cast
co_await (coroutines TS)
co_yield (coroutines TS)
default(1)
do
dynamic_cast
enum
export(1)
false
for
goto
import (modules TS)
int
module (modules TS)
namespace
noexcept (since C++11)
not_eq
operator
or_eq
protected
register(2)
requires (since C++20)
short
sizeof(1)
static_assert (since C++11)
struct(1)
synchronized (TM TS)
this
throw
try
typeid
union
using(1)
void
wchar_t
xor

SeAMK

Strong typing

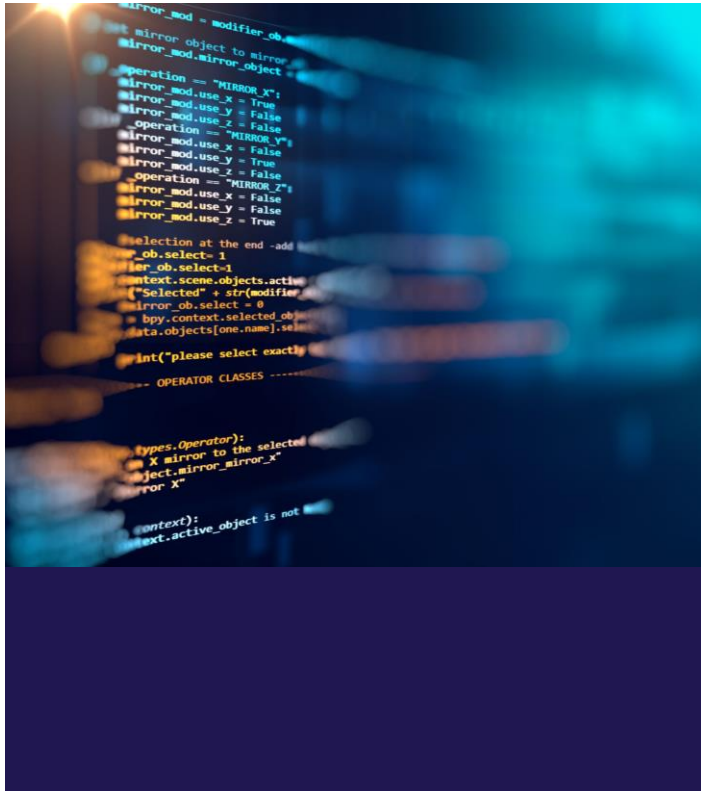
- Strongly typed unsafe language
  - Each type of data in program must have some defined type (int, double,…)
- Type conversions implicit or done by type casting
- Unsafe -> it is not prohibited if programmer wants to make unwise operations on typed variables



SeAMK

Manifest and inferred typing

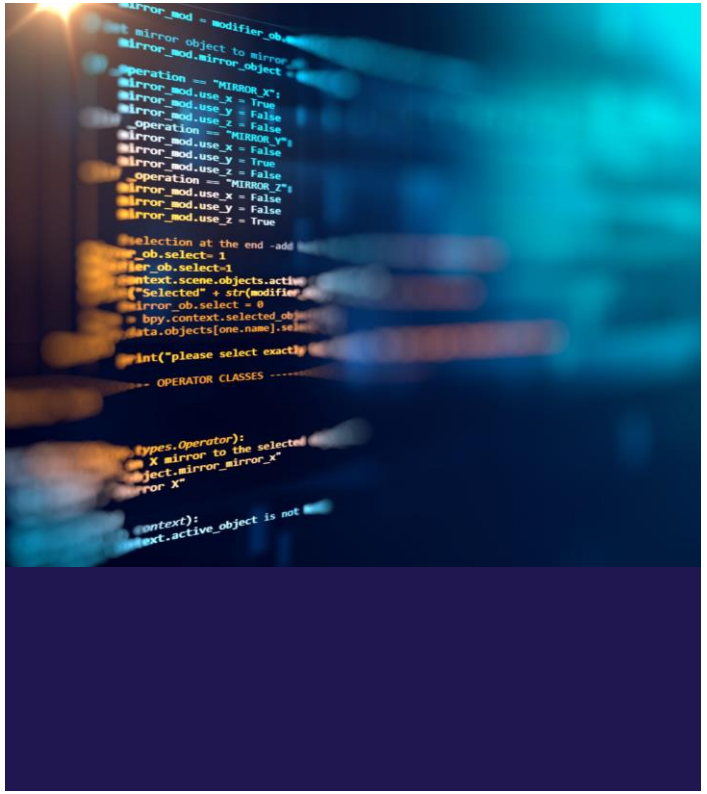
- Manifest typing means variables are given explicit type when declared
  - int value;
- Inferred typing: variable type based on context it is used (since C++ 11)
  - auto value = 3;
  - auto value = 3.14;



SeAMK

Static and dynamic type checking

- C++ allows type conversions to be checked either at compile-time or at run-time
- Flexibility allowed, but still most C++ type checking is static



SeAMK

Using different paradigms supported

- Support for procedural, generic and object-oriented programming paradigms
- Many others possible as well

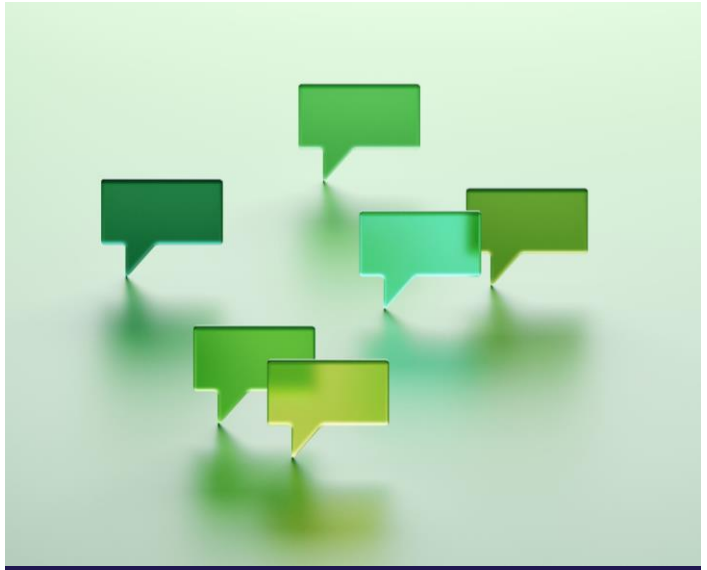
SeAMK

C++ benefits

- Fast running programs
- Efficiency and tools to handle memory
- Portability -> compiler found for most platforms
- Allows programmer lots of control (expects programmer to know what they are doing)
- Where to use: Anywhere where ultimate programmer control and efficiency needed

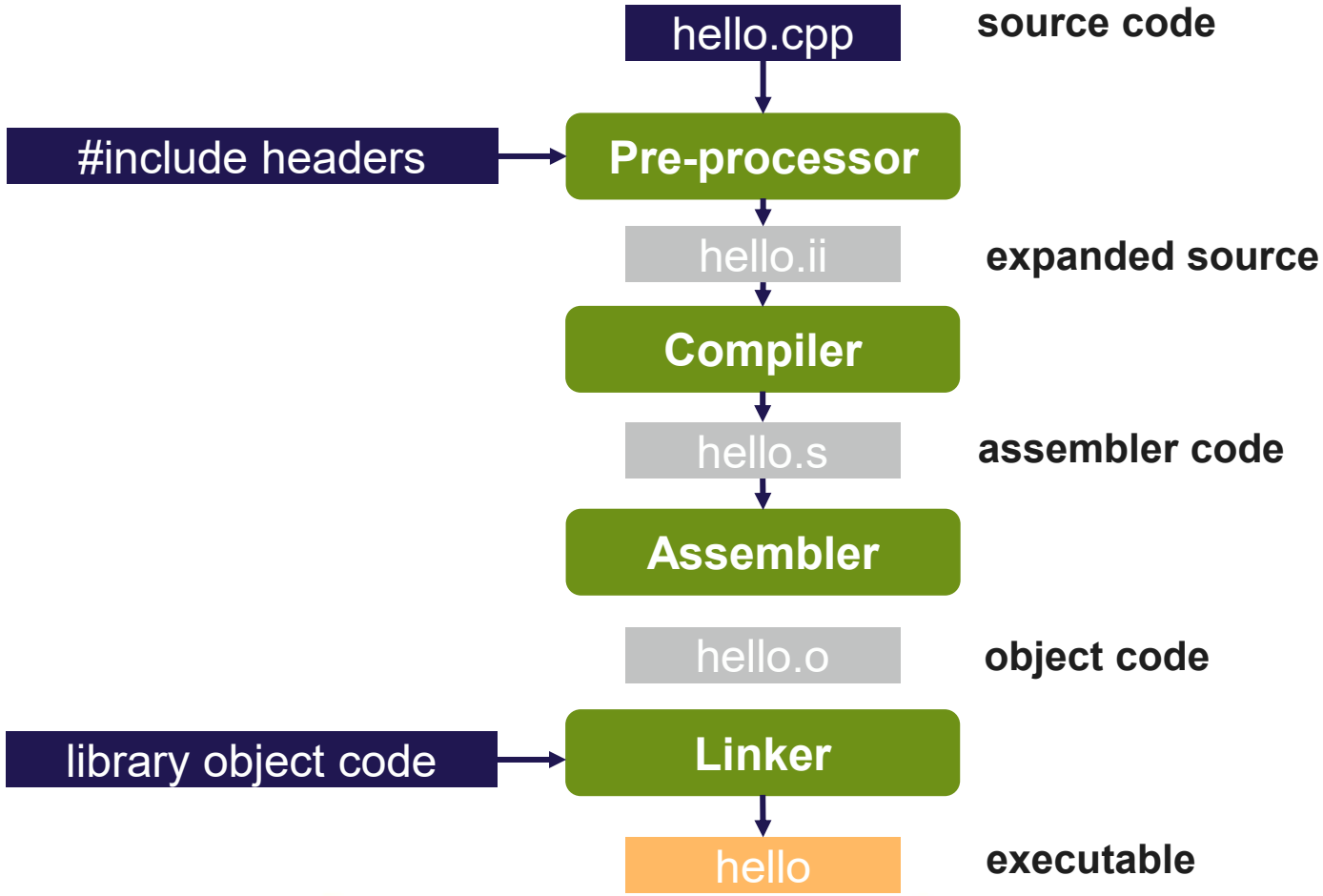
C++ style guide (Google)

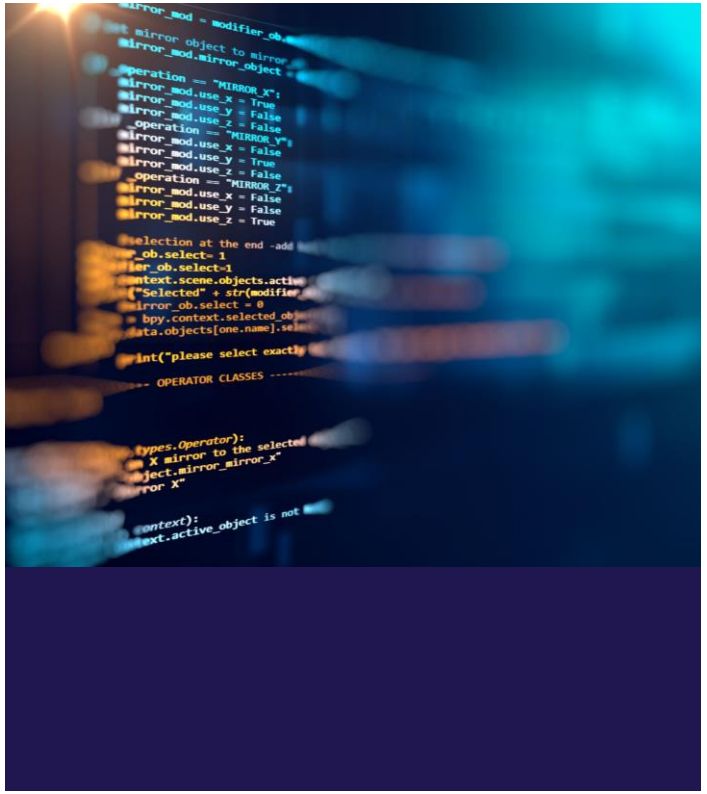
- Guide to manage complexity with C++
- Some goals:
  - “Optimize for the reader, not the writer”
  - “Be consistent with existing code”
  - “Avoid surprising or dangerous constructs”
  - “Avoid constructs that our average C++ programmer would find tricky or hard to maintain”



SeAMK

C++ compiler

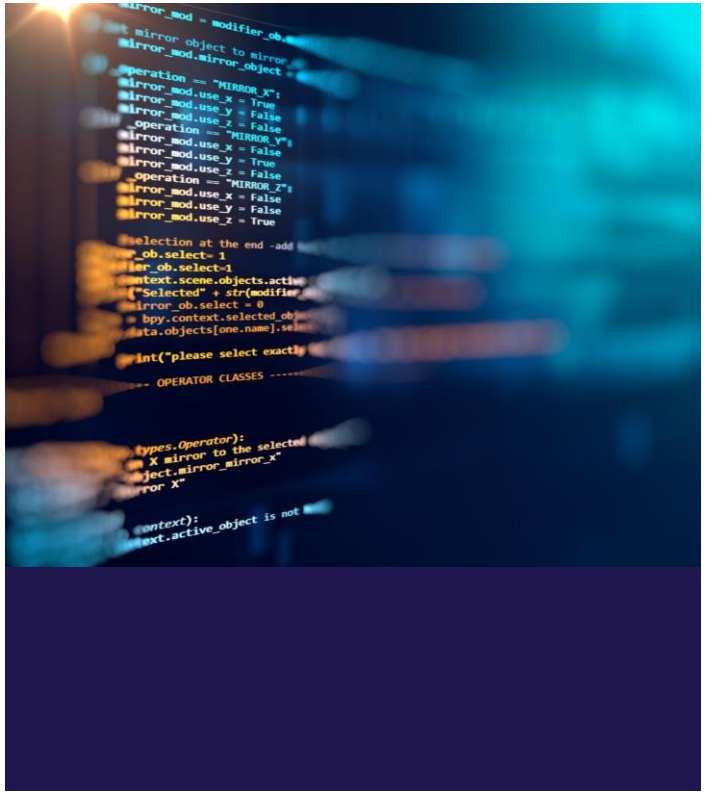




SeAMK

C++ compiler / IDE

- Visual Studio Code (Free option)
  - C++ programming with Visual Studio Code
  - Get Started with C++ and MinGW-w64 in Visual Studio Code
- g++ on Linux (some IDE needed)
  - g++ mycode.cpp -o myprogram.exe
  - g++ mycode.cpp -o myprogram.exe -std=c++11 -Wall –pedantic
- Visual Studio 2022



SeAMK

SEINÄJOEN AMMATTIKORKEAKOULU

SEINÄJOKI UNIVERSITY OF APPLIED SCIENCES